NATIONAL UNIVERSITY OF LESOTHO
Department of Computer Science — CS4433 Software Engineering

Moderated Learning Management System (LMS)

Final Project Report

Student Name: Lepota N.M
Student Number: 201904974
Course: CS4433 – Software Engineering
Evidence Basis: Consolidated Project Documents

Document Status and Evidence Basis: This report consolidates the Week 1 Project Management Plan, Week 2 SRS, Week 3 System Design, and Phases 5–8 documents. Findings distinguish between *Documented*, *Designed/Planned*, *Reported but Unverified*, and *Not Run/Not Measured*. The Week 4 starter contained basic logic skeletons; complete workflows, automated test execution, and deployment remain unverified against a running build.

Executive Report Structure & Table of Contents

1. Introduction & Project Overview	6. Verification & Validation
2. Feasibility Study & Project Planning	7. Quality Assurance, Security & Ethics
3. Requirements Engineering	8. Deployment & Maintenance
4. System Design & Architecture	9. Conclusion & Recommendations
5. System Implementation	Appendices A–E (Traceability, Issues, Rubric)

1. INTRODUCTION AND PROJECT OVERVIEW

1.1 Background & Operating Context

This project addresses the design of a Moderated Learning Management System (LMS) tailored for secondary-school environments. The system manages core academic workflows: course administration, assignment delivery, student submissions, marking, feedback, moderation, and result release. Designed for low-resource settings, it operates reliably on local hardware and offline networks without continuous internet access. Built with C++17, SQLite, and CMake, the application avoids high infrastructure overheads.

1.2 Problem Statement & Motivation

Educational institutions require tight control over assessment data to prevent premature grade exposure, inconsistent late penalties, and untraceable mark alterations. The central operational requirement dictates that lecturer-assigned marks remain hidden from students until formally reviewed, moderated, and released by the Head of Department (HoD). A dedicated local system ensures role separation and reliable persistence without requiring dedicated server infrastructure.

1.3 Project Aim and Objectives

Aim: To design and develop an offline-capable Moderated LMS enforcing role-based access, moderated result releases, and traceable audit logs.

- Deliver role-specific workflows for Students, Lecturers, HoDs, and Administrators.
- Enforce automated late-submission penalties and individual extension handling.
- Prevent student visibility of marks prior to HoD moderation and release.
- Maintain persistent database records and complete audit trails for all grade modifications.

1.4 Scope & User Roles

In Scope: Role authentication, user/course administration, assignment setup, extensions, multi-version submissions, marking, late penalties, HoD moderation, result release, and audit logging. *Out of Scope:* Online payments, video streaming, automated grading, plagiarism scanners, and web/mobile interfaces.

Role	Primary Responsibilities	Key System Privilege
Administrator	User management, department/course setup, enrolments	Full administrative system access
Lecturer	Assignment creation, individual extensions, marking, feedback	Submit initial marks and feedback
HoD	Department oversight, mark moderation, result release	Approve/return marks; authorize student release
Student	View assignments, submit/resubmit work, view released marks	Read-only access to own released results

2. FEASIBILITY STUDY AND PROJECT PLANNING

2.1 Feasibility Assessment

Dimension	Evaluation & Findings
Technical	C++17 and SQLite offer high efficiency on legacy hardware and operate completely offline. Multi-client file sharing on local network shares requires careful lock handling.
Operational	Role-aligned console workflows closely mirror school hierarchy, ensuring low training overhead.
Economic	Zero external software licensing costs; runs entirely on existing workstation equipment.
Legal/Ethical	Local storage protects student privacy. Complies with institutional requirements when access controls are enforced.
Schedule	Planned across a 7-week timeline using structured review gates to manage development risk.

2.2 Methodology & Planned Schedule

The project follows a sequential **Waterfall development model** coupled with **V-Model verification mapping**. Requirements and design artefacts are approved at formal quality gates prior to coding.

Phase / Week	Primary Focus	Key Deliverable / Review Gate
Week 1	Project Planning & Feasibility	WBS, Risk Register, Project Plan
Weeks 1–2	Requirements Engineering	SRS Document, Use Cases (**Gate 1 Approval**)
Weeks 2–3	System Design & Architecture	UML Diagrams, Database ERD (**Gate 2 Approval**)
Weeks 4–5	System Implementation	C++17 Core Engine & SQLite Persistence Layer
Weeks 5–6	Verification & Validation	Google Test Execution, Defect Reports (**Gate 3 Approval**)
Week 6	Deployment Preparation	Target Build Package & Verification Script
Week 7	Maintenance & Reporting	Change Request Evaluation & Final Consolidated Report

2.3 Risk Register

ID	Risk Description	Impact	Mitigation Strategy
R1	Incomplete implementation or build failure	High	Establish a known-good CMake build; enforce modular coding.
R2	Premature grade exposure before HoD approval	High	Service-layer enforcement of release status flag on all queries.
R3	Incorrect late-penalty calculation	Medium	Isolated unit testing of `PerDayPenaltyStrategy` logic.
R4	Database corruption or concurrency loss	High	Use SQLite transactions and maintain protected snapshot backups.

3. REQUIREMENTS ENGINEERING

3.1 Key Functional & Non-Functional Requirements

ID	Category	Requirement Description	Priority
FR-01	Security	Users shall authenticate using unique usernames and salted password hashes.	Must
FR-07	Assessment	Lecturers shall set assignment titles, descriptions, deadlines, and max marks.	Must
FR-14	Grading	System shall deduct 10% max mark per late day (up to 3 days); >3 days receives zero.	Must
FR-17/18	Workflow	HoDs shall moderate and release marks before student visibility is granted.	Must
FR-21	Audit	All grade updates must log changed-by user, timestamp, old mark, new mark, and reason.	Must
NFR-01	Performance	Core system functions must operate completely offline without internet dependence.	Must
NFR-07	Quality	Automated business logic unit tests must achieve at least 70% code coverage.	Must

3.2 Business Rules & Late Penalty Logic

Marks entered by lecturers remain in a `Marked` state. The HoD transitions status to `Approved` or `Returned`. Only upon explicit `Released` status can students access grades. Late penalties follow a deterministic calculation:

```
penaltyAmount = min(lateDays, 3) * 0.10 * maximumMark
if (lateDays > 3) finalScore = 0;
else finalScore = max(0.0, rawScore - penaltyAmount);
```

4. SYSTEM DESIGN

4.1 Architectural Design

The system utilizes a 4-tier **Layered Architecture** to achieve decoupling and maintainability:

- **Presentation Layer:** Role-based CLI menus enforcing user input sanitization.
- **Application Service Layer:** Orchestrates workflows, security, and transaction boundaries (e.g., `MarkingService`, `SubmissionService`).
- **Domain Layer:** Enforces business entity validation, state machines, and penalty strategies.
- **Persistence Layer:** Data Access Objects (DAOs) executing parameterised SQLite queries.

4.2 Class Design & Applied Design Patterns

The system incorporates key software engineering design patterns to maintain flexibility:

- **Strategy Pattern:** `ILatePenaltyStrategy` decouples calculation rules. `PerDayPenaltyStrategy` implements the default 10%/day rule, enabling alternative policies without refactoring core services.
- **Repository Pattern:** Decouples domain logic from SQL. Interfaces (e.g., `ISubmissionRepository`, `IMarkRepository`) allow mock testing without live database dependencies.

Documented Design Diagrams Summary		
Fig	**Diagram Type**	**Design Focus & Key Attributes**
1	Use-Case Diagram	Defines boundary actions for Student, Lecturer, HoD, and Admin roles.
2	Layered Architecture	Illustrates separation between CLI UI, Services, Domain Models, and SQLite.
3-5	Class Diagram Views	Detailed entity structures, inheritance (`User` subtypes), and repository interfaces.
6	State Machine Diagram	Models submission transitions: *Submitted* → *Marked* → *Moderated* → *Released*.
7-9	Activity & Sequence	Swimlane interactions establishing mandatory HoD release gates before grade display.
10-11	ERD Schemas	Relational structure in 3rd Normal Form (3NF) supporting multi-version submission data.

5. SYSTEM IMPLEMENTATION

5.1 Technical Stack & Build Setup

The target application is implemented using **C++17**, utilizing modern STL containers, standard chrono utilities for timestamp evaluation, and SQLite3. Build automation is managed via **CMake (v3.15+)**.

5.2 Relational Database Schema (SQLite DDL)

```
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL CHECK(role IN ('Student', 'Lecturer', 'HoD', 'Admin'))
);

CREATE TABLE assignments (
    assignment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    deadline TEXT NOT NULL,
    max_mark REAL NOT NULL,
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

CREATE TABLE submission_versions (
    version_id INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id INTEGER NOT NULL,
    version_number INTEGER NOT NULL,
    file_path TEXT NOT NULL,
    submitted_at TEXT NOT NULL,
    is_late INTEGER NOT NULL,
    FOREIGN KEY (submission_id) REFERENCES submissions(submission_id)
);

CREATE TABLE marks (
    mark_id INTEGER PRIMARY KEY AUTOINCREMENT,
    version_id INTEGER NOT NULL UNIQUE,
    raw_score REAL NOT NULL,
    penalty_amount REAL NOT NULL,
    final_score REAL NOT NULL,
    feedback TEXT,
    marked_by INTEGER NOT NULL,
    FOREIGN KEY (version_id) REFERENCES submission_versions(version_id)
);

CREATE TABLE mark_audit_logs (
    audit_id INTEGER PRIMARY KEY AUTOINCREMENT,
    mark_id INTEGER NOT NULL,
    changed_by INTEGER NOT NULL,
    old_score REAL NOT NULL,
    new_score REAL NOT NULL,
    reason TEXT NOT NULL,
    changed_at TEXT NOT NULL
);
```

6. VERIFICATION AND VALIDATION

6.1 Testing Strategy

Verification follows the V-Model testing hierarchy. Testing activities are categorized into Unit Tests (Google Test execution of logic components), Integration Tests (SQLite repository persistence checks), and System Use-Case Tests.

6.2 Test Suite Specification & Execution Status

ID	Test Target	Input / Action Scenario	Expected Outcome	Status
TC-01	Auth Service	Submit correct / incorrect credentials	Valid login granted; invalid rejected	Designed
TC-02	Role Security	Student attempts lecturer menu access	Access blocked; security error logged	Designed
TC-03	Submission	Student uploads assignment before deadline	Status `On-Time`; penalty = 0.0	Designed
TC-04	Late Penalty	Submission 2 days past deadline	Status `Late`; 20% penalty applied	Designed
TC-05	Extreme Late	Submission >3 days past deadline	Submission recorded; final mark = 0.0	Designed
TC-06	Extensions	Submit after original but before extended date	Status `On-Time`; 0 penalty applied	Designed
TC-07	Moderation Gate	Student queries marked unreleased grade	Grade hidden; *"Pending Moderation"*	Designed
TC-08	Result Release	Student queries grade after HoD release	Final mark and feedback displayed	Designed
TC-09	Audit Trail	Lecturer updates an existing score	Old/new scores & reason logged to audit	Designed

Note: Test execution output logs and automated line-coverage percentages remain unverified as testing was not executed against the compiled starter codebase.

7. QUALITY ASSURANCE, SECURITY AND PROFESSIONAL ISSUES

7.1 Security Enforcement

- **Password Security:** Complies with NFR-03 by mandating salted hashing algorithms (e.g., bcrypt) rather than plaintext storage.
- **SQL Injection Prevention:** Mandates parameterised bindings across all Data Access Objects (NFR-04).
- **Data Privacy:** Enforces role-level filtering so students cannot access peer records.

7.2 Professional & Legal Ethics

The system handles high-stakes evaluation records. Developers maintain a professional obligation to ensure score accuracy, transparent late-penalty rules, and immutable audit logs. Strict access management guarantees compliance with education record privacy standard practices.

8. DEPLOYMENT & MAINTENANCE PLAN

8.1 Local Target Deployment

Deployment entails creating a release build using CMake, executing SQL initialisation DDL files to establish the local SQLite database file, and pre-populating administrative root accounts.

8.2 Maintenance Strategy & Change Requests (CR1)

Maintenance focuses on code bug fixes and incorporating prospective change requests. Change Request 1 (CR1)—proposing class-wide deadline extensions—was evaluated: it requires schema adjustments to the `assignments` table and updated service layer deadline-resolution logic prior to implementation.

9. CONCLUSION AND RECOMMENDATIONS

9.1 Conclusion

This report has presented the comprehensive design and document-based evaluation of the Moderated Learning Management System. The design documents establish a robust architecture, sound relational database model, clear role separation, and rigorous design pattern application. However, physical repository analysis indicates that the implementation remained at a starter foundation stage. Full verification requires completed coding, automated test execution, and deployment validation.

9.2 Key Recommendations

1. **Complete Business Workflows:** Implement missing CLI menu dispatchers and core service implementations for marking, moderation, and releases.
2. **Schema Synchronization:** Reconcile database table naming discrepancies with class models as detailed in Appendix C.
3. **Execute Automated Testing:** Run the Google Test suite, log execution records, and verify line coverage against the 70% requirement.
4. **Security Audit:** Validate parameterised SQL binding and salted password hashing prior to operational deployment.

REFERENCES

- Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
- SQLite Consortium. (2026). *SQLite Documentation and Interface Guides*. https://www.sqlite.org/docs.html

APPENDIX A – REQUIREMENTS TRACEABILITY MATRIX

Req ID	SRS Description	Design Component	Test ID	Status
FR-01	User authentication	`AuthService` / `users` table	TC-01	Unverified
FR-07	Assignment creation	`AssignmentService` / `assignments`	TC-03	Unverified
FR-14	Late-penalty deduction	`PerDayPenaltyStrategy`	TC-04, TC-05	Unverified
FR-17/18	HoD moderation & release	`ModerationService` / `ReleaseRecord`	TC-07, TC-08	Unverified
FR-21	Mark audit logging	`MarkAuditLog` / `IAuditRepository`	TC-09	Unverified

APPENDIX B & C – SUMMARY OF TECHNICAL ISSUES

Area	Identified Issue / Discrepancy	Remediation Requirement
Naming Discrepancy	SQL DDL uses snake_case table names; class diagrams use PascalCase entity names.	Implement explicit DAO mapping layer to reconcile schema and class entities.
Class Model Omission	`IAuditRepository` interface is present in View C, but concrete SQLite class is missing.	Add explicit `SqliteAuditRepository` class block to UML class diagrams.
Codebase Gap	Starter code lacks menu router and workflow service bindings.	Complete core implementation of application service layers.

APPENDIX D & E – COMPLIANCE CHECKLIST & MISSING EVIDENCE

Evaluation Item	Compliance Assessment	Missing Evidence / Action Required
Rubric Alignment	Fully compliant with report structure requirements	Requires physical test execution logs to verify implementation depth.
Build & Test Logs	Pending verification	Requires CMake build outputs and Google Test XML test reports.
Diagram Artefacts	Embedded images reviewed	Requires high-resolution vector source diagram files (.drawio / .puml).